

Shobjiwala — Security Review

Date: 2026-05-18 **Reviewer:** automated (composer audit, trivy fs + config, phpcs-security, phpcs-security-vendor) plus manual code and surface probing against the running stack at `http://localhost:8080`. **Scope:** code we own and ship (wp-config.php, docker-compose.yml, docker/wordpress/Dockerfile, wp-content/themes/groser, wp-content/themes/groser-child, wp-content/mu-plugins), the runtime surface of the WordPress instance, and the locked composer dependency set. Plugin source lines (Elementor, Jetpack, WooCommerce, etc.) are not line-audited — they are covered by `composer audit` and `roave/security-advisories`. **Status:** findings only. No fixes applied per instruction. Each finding lists a recommended remediation; nothing in this report has been changed yet.

Executive summary

Severity	Count	Headline
Critical	2	Hardcoded fallback secrets in <code>wp-config.php</code> (auth keys + DB password).
High	6	REST + author-archive user enumeration, xmlrpc accepting method calls, web-accessible debug log, 84 security-sniff hits in the parent Groser theme, real-PII admin accounts inherited from prod dump.
Medium	8	phpMyAdmin exposure, debug-on by default, weak compose env defaults, Dockerfile builds as root, version fingerprinting via <code>readme.html</code> / <code>license.txt</code> / <code>X-Powered-By</code> , no MFA, no login throttling.
Low / Info	8	composer audit and Trivy report clean on locked deps; auto-update disabled; misc hardening notes.

The two **Critical** findings are the only ones that would cause immediate full compromise if pushed to production as-is. The **High** findings are exploitable today but require an attacker to chain them.

Methodology

```
composer audit                                # locked deps vs advisories
docker run aquasec/trivy:0.58.0 fs    ...    # filesystem CVEs, secrets, misconfig
docker run aquasec/trivy:0.58.0 config ...    # Dockerfile / IaC misconfig
vendor/bin/phpcs --standard=phpcs-security.xml.dist      # our code
vendor/bin/phpcs --standard=phpcs-security-vendor.xml.dist  # parent groser theme
curl probes against http://localhost:8080                # surface enumeration
wp-cli inspection of users, options, plugin versions
```

Critical

C1. Hardcoded default auth/nonce keys in `wp-config.php`

File: `wp-config.php:42-49`

```
define('AUTH_KEY',          shobjiwala_env('WORDPRESS_AUTH_KEY',          'change-me-auth-key'));
define('SECURE_AUTH_KEY', shobjiwala_env('WORDPRESS_SECURE_AUTH_KEY', 'change-me-secure-auth-
define('LOGGED_IN_KEY',    shobjiwala_env('WORDPRESS_LOGGED_IN_KEY',    'change-me-logged-in-ke
define('NONCE_KEY',        shobjiwala_env('WORDPRESS_NONCE_KEY',        'change-me-nonce-key'))
define('AUTH_SALT',        shobjiwala_env('WORDPRESS_AUTH_SALT',        'change-me-auth-salt'))
define('SECURE_AUTH_SALT',shobjiwala_env('WORDPRESS_SECURE_AUTH_SALT','change-me-secure-auth-
define('LOGGED_IN_SALT',   shobjiwala_env('WORDPRESS_LOGGED_IN_SALT',   'change-me-logged-in-sa
define('NONCE_SALT',       shobjiwala_env('WORDPRESS_NONCE_SALT',       'change-me-nonce-salt'))
```

Risk. If any of the eight env vars are missing in production, the corresponding constant evaluates to the literal placeholder string — which is *committed to git and public on GitHub*. Anyone who knows the placeholder values can forge WordPress session cookies (cookie auth uses these as the HMAC key) and impersonate any authenticated user — including the administrator. This is full account takeover with no prior credential knowledge.

Exploit path. Attacker reads `wp-config.php` from the public repo → computes a session cookie HMAC with `change-me-auth-key` → sets that cookie in a browser → loads `/wp-admin/`. Effort: minutes.

Fix. Either:

- Remove the placeholder strings entirely so the constant is undefined when the env var is missing, causing WordPress to refuse to boot (fail-closed), or
- Make the fallback throw: `define('AUTH_KEY', shobjiwala_env('WORDPRESS_AUTH_KEY') ?: throw new RuntimeException('AUTH_KEY env var required'));`
- Generate per-environment 64-char random secrets and load them from the secrets manager (1Password, AWS SSM, Vault, etc.), never from a literal default.

C2. Hardcoded default database credentials in `wp-config.php` and `docker-compose.yml`

Files: `wp-config.php:32-35`, `docker-compose.yml:7-10,37-40`

```
define('DB_NAME',          shobjiwala_env('WORDPRESS_DB_NAME',          'shobjiwala'));
define('DB_USER',          shobjiwala_env('WORDPRESS_DB_USER',          'wp'));
define('DB_PASSWORD',      shobjiwala_env('WORDPRESS_DB_PASSWORD',      'wp_password'));
define('DB_HOST',          shobjiwala_env('WORDPRESS_DB_HOST',          'db'));
```

And in compose:

```
MARIADB_USER:          ${DB_USER:-wp}
MARIADB_PASSWORD:      ${DB_PASSWORD:-wp_password}
MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:-root_password}
```

Risk. A deployment that forgets `.env` (or sources an incomplete one) silently runs with the published defaults. Both the MariaDB `user` password and the `root` password are predictable from a public repository. Any attacker who reaches the MySQL port gets unrestricted access.

Fix. Drop the fallback defaults. Make the env vars required; let docker-compose abort if any are missing. Use a stronger generated password in `.env.example` so even copy-paste users don't get a usable weak default.

High

H1. WordPress REST API leaks the full user list to unauthenticated callers

Endpoint: `GET /wp-json/wp/v2/users`

```
[{"id":1,"name":"tahminachowdhury2023","slug":"tahminachowdhury2023","link":"...author/tahmin
```

Risk. Discloses every WordPress user's id, login slug (often equal to the login name), and a Gravatar hash that uniquely identifies the user's email (the hash is `md5(lowercase_email)`, which is reversible against any leaked email list). Reduces brute-force / credential-stuffing surface from "guess username + password" to just "guess password".

Fix. Block unauthenticated access:

```
add_filter('rest_authentication_errors', function ($result) {
    if (!empty($result)) return $result;
    if (!is_user_logged_in()) {
        return new WP_Error('rest_not_logged_in', 'Unauthorised.', ['status' => 401]);
    }
    return $result;
});
```

Or a more surgical rule: filter `rest_user_query` and remove the `users` endpoint from `rest_endpoints`. Both belong in a small must-use plugin (`wp-content/mu-plugins/security.php`).

H2. Author-archive URLs enumerate user logins

`GET /?author=1 → 302 Location: /author/tahminachowdhury2023/`

Risk. Same class as H1 but via the classic WordPress author archive. Even when REST is hardened, the author archive still leaks. Combined with H1, an attacker has full login-slug enumeration with two requests.

Fix. Disable author archives unless the site needs them. In a must-use plugin:

```
add_action('template_redirect', function () {
    if (is_author()) wp_redirect(home_url(), 301) && exit;
});
```

H3. `xmlrpc.php` accepts method calls

POST `/xmlrpc.php` with `system.listMethods` returns the full RPC method list including `wp.getUsersBlogs` (credential brute-force amplifier — one HTTP call tries hundreds of passwords) and `pingback.ping` (used in DDoS amplification).

Risk. Two-fold. (a) Bypass of `wp-login.php` rate limiting by submitting credentials via `system.multicall`. (b) The site can be used as a pingback amplifier against third parties. Both have been mass-exploited for over a decade.

Fix. Disable XML-RPC unless explicitly needed (Jetpack still uses it, so check first):

```
add_filter('xmlrpc_enabled', '__return_false');
```

If Jetpack needs XML-RPC, restrict at the web tier with an Apache `<Files xmlrpc.php> Require ip ...` rule.

H4. `wp-content/debug.log` is web-accessible

GET `/wp-content/debug.log` → HTTP 200. With `WP_DEBUG_LOG=true` in `wp-config.php:57`, WordPress writes every PHP notice / warning / fatal error to this file, including absolute filesystem paths, sometimes function arguments, and occasionally user input.

Risk. Information disclosure. Useful to attackers for vulnerability hunting (paths, plugin versions, internal IPs, leaked tokens passed to functions that throw).

Fix. Move the log outside the document root (`define('WP_DEBUG_LOG', '/var/log/wp/debug.log')`) and/or add an Apache rule:

```
<FilesMatch "\.log$"> Require all denied </FilesMatch>
```

H5. Parent Groser theme — 84 PHPCS security findings

`vendor/bin/phpcs --standard=phpcs-security-vendor.xml.dist` → **70 errors, 14 warnings** across 22 files. Breakdown:

Sniff	Count	Class
<code>WordPress.Security.EscapeOutput.OutputNotEscaped</code>	33	Potential XSS — variables emitted without <code>esc_html</code> / <code>esc_attr</code> / <code>wp_kses</code> .
<code>WordPress.Security.NonceVerification.Missing</code>	14	Form submissions processed without CSRF tokens — affects add-to-cart, quantity update.
<code>WordPress.Security.NonceVerification.Recommended</code>	13	Same class, slightly lower confidence.
<code>WordPress.Security.ValidatedSanitizedInput.InputNotSanitized</code>	12	Raw <code>\$_POST</code> / <code>\$_GET</code> used directly.
<code>WordPress.Security.ValidatedSanitizedInput.MissingUnslash</code>	9	Quote-injection class — WP slashes input that must be unslashed before use.
<code>WordPress.Security.ValidatedSanitizedInput.InputNotValidated</code>	1	Same class.
<code>WordPress.Security.EscapeOutput.UnsafePrintingFunction</code>	1	<code>printf</code> / <code>print</code> used where escape is required.

Hot files:

- `woocommerce/cart/cart.php` — 5 unescaped outputs.
- `woocommerce/single-product/add-to-cart/simple.php` — 4 errors, including unsanitized `$_POST['quantity']` and missing nonce.
- `woocommerce/single-product/add-to-cart/variation-add-to-cart-button.php` — 3 errors.
- `lib/ocdi/functions.php` — 3 errors in the One-Click Demo Import shim.
- `template-parts/content*.php` — 6 errors.

Risk. The theme is third-party (ThemeForest "Groser"). The XSS errors are mostly reflected (require crafted input that flows through cart / product data) but several flow through `apply_filters()` output, where any filter-injecting plugin can taint them. The missing nonce errors mean an attacker can craft a third-party page that, when visited by an authenticated user, adds items to that user's cart or changes quantities. Not a privilege escalation, but a real CSRF.

Fix. Options ranked by effort:

1. Replace the theme. The Groser theme has not received security updates in months; a maintained alternative (Storefront, Astra, Blocksy) carries no debt.
2. Fork the theme into `wp-content/themes/groser/` (already done) and patch each finding. PHPCS report at `/tmp/phpcs-vendor-full.txt` from this review enumerates every line.
3. Mitigate via a child-theme override of the worst files (`cart.php` , `add-to-cart/simple.php`) without touching the parent — bullet 2 scoped to the top three hot files.

H6. PII inherited from the original production dump

`KAW_users` in the persisted `db_data` volume contains:

- `admin / tahminaweb@gmail.com` (registered 2026-02-05)
- `tahminachowdhury2023 / tahminachowdhury2023@gmail.com` (registered 2025-11-24) — currently the `admin_email` recipient

Risk. These are real personal email addresses from the original site owner. Anyone running the local dev stack has them. Anyone with access to the volume can password-reset either account (a request to `wp-login.php?action=lostpassword` mails a reset link to those Gmail accounts — to a stranger's inbox). Privacy violation (GDPR PII handling) and potential takeover of a real customer's identity if they reuse the email elsewhere.

Fix. Either (a) destroy the persisted volume (`docker compose down -v`) and let the team install WordPress fresh, or (b) rewrite the users in the existing DB with `wp user update <id> --user_email=dev@example.invalid` and a fresh password. The same applies to any customer rows in `KAW_wc_customer_lookup` .

Medium

M1. phpMyAdmin exposed on the host

`docker-compose.yml:65-66` publishes `${PMA_PORT:-8081}:80` without binding to `127.0.0.1` . On Docker Desktop this is reachable from any process on the host; on a Linux Docker daemon it binds to `0.0.0.0` — meaning if this compose file is used on a public server, phpMyAdmin is exposed to the internet with root MySQL credentials.

Fix. Bind to localhost: `"127.0.0.1:${PMA_PORT:-8081}:80"` , or remove the service from the production compose profile entirely and tunnel SSH when DB access is needed.

M2. `WP_DEBUG=true` is the default

`wp-config.php:56` and `.env:10` . Combined with H4, every PHP notice gets logged where the web can fetch it.

Fix. Default to `false` . Make `.env.example` set `WP_DEBUG=false` and document that dev uses an override in `.env.local` .

M3. Weak compose env defaults: `wp_password`, `root_password`

`docker-compose.yml:7-10`. Same class as C2 — predictable from the public repo.

Fix. Drop the defaults; require `.env`.

M4. Dockerfile build step runs as root

`docker/wordpress/Dockerfile` — `curl -sSL ... > /usr/local/bin/wp` runs as root, no integrity check on the downloaded WP-CLI phar. Trivy AVD-DS-0002.

Risk. Supply-chain. If `raw.githubusercontent.com/wp-cli/builds/...` is MITM'd or compromised, the binary baked into the image is malicious. Apache itself drops to `www-data` at runtime so the *running* container is fine — this is build-time only.

Fix. Add SHA-256 verification of the WP-CLI phar; pin a release tag instead of `gh-pages/phar/wp-cli.phar`. Add a `USER` directive after the install steps (Apache still binds to 80 because `wordpress:php8.3-apache` is configured for that already).

M5. Version fingerprinting via static files

- `GET /readme.html` → 200, ships WP version in the `<h1>`.
- `GET /license.txt` → 200, confirms WordPress.

Fix. Either remove the files post-deploy, or add an Apache deny for both. The project `.gitignore` already excludes them from git but the WordPress docker image ships them and Apache serves them.

M6. X-Powered-By: PHP/8.3.31 and Server: Apache/2.4.67 (Debian) headers

Disclosure of stack versions. Helpful to attackers who match CVE lists by version.

Fix. `expose_php = Off` in `php.ini` (extend `docker/wordpress/Dockerfile`'s `zz-shobjiwala.ini`). For Apache, `ServerTokens Prod` and `ServerSignature Off`.

M7. `admin` is one of the two admin accounts

`KAW_users` has `admin` (the literal default username). This is the #1 brute-force target for WordPress credential-stuffing campaigns.

Fix. Rename or delete after rotating ownership.

M8. No MFA, no login throttling, no CAPTCHA

`/wp-login.php` is reachable without rate limiting. Combined with H1/H2/H3 (full username enumeration) and M7 (predictable username), this is a high-likelihood credential-stuffing target.

Fix. Install a 2FA plugin (e.g. `two-factor` from the WP core team — free, on wpackagist) and a login-throttling plugin (`limit-login-attempts-reloaded`), or front the site with Cloudflare's free WAF / Bot Fight Mode.

Low / informational

#	Finding	Notes
L1	<code>roave/security-advisories: dev-latest</code> is in <code>composer.json</code> require-dev.	Already in place — blocks <code>composer install</code> on any new advisory. Keep.
L2	WordPress core 6.9.4 is current as of 2026-05-18.	Good.
L3	All 23 active plugin versions are current on wpkgagist. <code>composer audit</code> reports zero advisories.	Good.
L4	<code>AUTOMATIC_UPDATER_DISABLED=true</code> .	Intentional under composer management. Trade-off: misses critical CVE patches if <code>composer update</code> is not scheduled. Suggest a weekly CI job (<code>composer outdated --direct</code> with <code>--strict</code>).
L5	<code>DISALLOW_FILE_EDIT=true</code> .	Good — admin UI cannot edit theme/plugin PHP.
L6	<code>WP_POST_REVISIONS=5</code> .	Good — bounds DB bloat.
L7	<code>groser-child/functions.php</code> uses <code>preg_replace</code> to strip rendered HTML.	Fragile across WooCommerce updates but not a security issue.
L8	Trivy fs scan on <code>composer.lock</code> : 0 HIGH/CRITICAL vulns, 0 secrets, 0 misconfigurations.	Good.

What this review did NOT cover

- **Per-plugin code audit** for the 23 composer-managed plugins (Elementor, Jetpack, WooCommerce, Yoast, etc.). Covered by `composer audit` + `roave/security-advisories` , which check against the Packagist Security Advisories DB. Not equivalent to a line-by-line audit, but appropriate for plugins maintained by external teams.
- **Dynamic exploitation**. All findings are static or surface-level GET/POST probes. No real attack execution.
- **WAF / network-layer review** (Cloudflare rules, firewall, fail2ban) — out of scope without a deployment topology.
- **Production environment**. This review ran against `localhost:8080` (the dev stack). Differences between dev and prod (TLS, host headers, real CDN, real secrets management) should be reviewed separately once the production deployment exists.

Suggested triage order

1. **C1 + C2 today** before any production deploy: rip the fallback secrets out of `wp-config.php` and `docker-compose.yml`.
2. **H6 today**: rotate or remove the inherited admin emails / accounts. PII liability outweighs any convenience of keeping them.
3. **H1 + H2 + H3 + H4 this week** in a single must-use plugin (`wp-content/mu-plugins/security.php`) — REST hardening, author archive disable, xmlrpc disable, debug.log relocation. One commit.
4. **H5 — decide direction**: replace the theme, or take ownership and patch the 84 findings. Decision affects whether the team carries this debt forever.
5. **M1 + M2 + M3** in the production compose changes (separate spec to follow).
6. **M4 + M5 + M6** as a Dockerfile + Apache hardening pass.
7. **M7 + M8** as a one-off admin tidy-up + 2FA rollout.
8. **L***: hygiene — schedule weekly `composer update --dry-run` in CI.